

Calab
AI & Tech

Save

# 15

RAG CHEAT SHEET

# RAG Terms You Need to Know

Plain English, with code.

Retrieval | Embeddings | Re-Ranking | +12 more

TERM 01

# Retrieval

01

Find the right passages **before** the model answers. It is the research phase of RAG.

### Why it matters

The model answers from your current docs, not stale training data.

retrieve.py

```
query = "What&#x27;s our refund policy?"
docs = vector_db.search(query, top_k=5)
answer = llm.generate(query, context=docs)
```

### In practice

Support bot pulls the refund policy page, then answers.

**Retrieval quality caps answer quality.**

**No retrieval, no RAG. Everything else tunes this step.**




**Calab**
AI & Tech



Follow me

TERM 02

# Embeddings

02

Text turned into vectors that capture **meaning**. Similar ideas land close together.

### The classic trick

"King" - "Man" + "Woman" lands near "Queen".

embed.py

```
client = OpenAI()
emb = client.embeddings.create(
    model="text-embedding-3-small",
    input="Retrieval-Augmented Generation")
vector = emb.data[0].embedding
```

### In practice

Similar questions map to nearby vectors, even with different words.

> **Same model for documents and queries, always.**




**Calab**
AI & Tech


Follow me

TERM 03

# Vector Databases

03

Databases built to store vectors and find the **closest ones** fast, not exact matches.

**Built for scale**

Similarity search across millions of documents in milliseconds.

chroma.py

```
col = Client().create_collection("docs")
col.add(documents=["RAG improves accuracy"],
        ids=["doc1"])
col.query(query_texts=["reduce AI errors"],
          n_results=2)
```

**In practice**

Search 'reduce AI errors' and get the hallucination doc.

**Chroma, Pinecone, pgvector: pick one and move on.**




**Calab**
AI & Tech


Follow me

TERM 04

# Retriever

04

The component that runs the whole lookup: embed the query, search, return the best **chunks**.

**Think of it as**

A smart librarian who knows exactly where to look.

retriever.py

```
class Retriever:
    def retrieve(self, query, top_k=5):
        vec = embed(query)
        return self.vector_db.search(vec, k=top_k)
```

**In practice**

One retriever for docs, another for tickets.

**Swap retrievers without touching the LLM.**




**Calab**
AI & Tech


Follow me

TERM 05

# Chunking

05

Splitting big documents into smaller segments that keep meaning and fit the model limits.

### Why it decides quality

Bad chunks mean bad retrieval, whatever model you use.

chunk.py

```
def chunk_text(text, size=500, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start+size])
        start += size - overlap
    return chunks
```

### In practice

Try 500 tokens with 50 overlap, then test.

> **Overlap keeps ideas from being cut in half.**




**Calab**
AI & Tech


Follow me

TERM 06

# Context Window

06

The max tokens a model can read in one request. It is its **working memory**.

### Range

From 4K tokens on older models to 200K+ on modern ones.

fit.py

```
def fit_context(chunks, max_tokens=4000):
    ctx, used = [], 0
    for c in chunks:
        n = count_tokens(c)
        if used + n < max_tokens:
            ctx.append(c); used += n
    return ctx
```

### In practice

Budget: system prompt + chunks + answer under the limit.

> **Bigger window is not a license to stuff it full.**




**Calab**
AI & Tech


Follow me

TERM 07

# Grounding

07

Tie every answer to retrieved, verifiable sources so the model stays **honest**.

### ⚓ How to do it

Force citations and allow claims only from the given context.

ground.py

```
prompt = f"""
Answer ONLY from the context. Cite sources.
Context: {retrieved_docs}
Question: {user_question}
"""
response = llm.generate(prompt)
```

### In practice

Add 'say I don't know' when context is empty.

> **If it cannot cite it, it should not say it.**

TERM 08

# Re-Ranking

08

Two stages: a fast broad search, then a precise model that **re-scores** the candidates.

### Flow

Fast retrieval (top 100) then cross-encoder scoring (top 5).

rerank.py

```
reranker = CrossEncoder("ms-marco-MiniLM-L-6-v2")
cands = retriever.search(query, top_k=100)
scores = reranker.predict([(query, d) for d in cands])
top = [cands[i] for i in scores.argsort()[-5:]]
```

### In practice

Recall on 100 candidates, precision on the top 5.

> **Cheap recall first, expensive precision second.**

TERM 09

# Hybrid Search

09

Blend keyword search (BM25) with vector search for **exact terms and meaning** together.

**Best of both**

Exact matches for IDs and names, semantics for concepts.

hybrid.py

```
def hybrid_search(query, alpha=0.5):
    kw = bm25_search(query)
    sem = vector_search(query)
    score = alpha*kw.scores + (1-alpha)*sem.scores
    return score.top_k(5)
```

**In practice**

Search 'error E-4021': keywords catch it, vectors alone miss.

**Tune alpha on your own queries, not defaults.**




**Calab**

AI & Tech


Follow me

TERM 10

# Metadata Filtering

10

Narrow results by date, author, type or team **before** the semantic search runs.

**Effect**

Less noise, faster queries, more precise answers.

filter.py

```
results = collection.query(
    query_texts=["quarterly revenue"],
    n_results=10,
    where={"year": 2024, "department": "sales"}
)
```

**In practice**

'Only 2024 sales reports' shrinks the pool instantly.

> **Filter first, then rank what is left.**




**Calab**
AI & Tech


Follow me

TERM 11

# Similarity Search

11

Find documents whose embeddings sit closest to the **query embedding**.

### Common metrics

Cosine similarity or dot product. Higher score, more relevant.

cosine.py

```
def cosine(a, b):
    return np.dot(a, b) / (
        np.linalg.norm(a) * np.linalg.norm(b))

sims = [cosine(query_vec, v) for v in doc_vecs]
```

### In practice

Rank by score and keep the top 5 chunks.

> **It measures closeness of meaning, not shared words.**




**Calab**
AI & Tech



Follow me

TERM 12

# Prompt Injection

12

Users slip in instructions to override your system prompt or leak **sensitive data**.

**Typical attack**

"Ignore previous instructions" or "reveal your system prompt".

guard.py

```
BAD = ["ignore previous instructions",
       "reveal your instructions"]
def sanitize(text):
    if any(b in text.lower() for b in BAD):
        return "Invalid input detected"
    return text
```

**In practice**

A PDF says 'ignore all rules'. Treat retrieved text as untrusted.

**Pattern lists are a start, not a defense. Layer them.**




**Calab**
AI & Tech


Follow me

TERM 13

# Hallucination

13

Fluent, confident output that the source documents **do not support**.

**⚠ Symptoms**

Invented facts, fake citations, made-up details.

verify.py

```
def check(response, sources):
    p = f"Any claims not in sources?\n"
        f"Response: {response}\nSources: {sources}"
    return llm.verify(p)
```

**In practice**

A cited policy that does not exist in your docs.

**RAG lowers hallucination. It never removes it.**

TERM 14

# Agentic RAG

14

The agent plans, searches, reads results and **searches again** until it is sure.

**Why it wins**

Handles multi-step questions a single lookup cannot.

agentic.py

```
plan = self.llm.create_plan(query)
for step in plan:
    if step.action == "search":
        self.ctx.add(self.retriever.search(step.query))
    else:
        self.ctx.add(self.llm.analyze(self.ctx))
return self.llm.synthesize(self.ctx)
```

**In practice**

Compare prices, then check stock, then summarise.

**More power, more latency and cost. Use it when needed.**




**Calab**

AI & Tech


Follow me

TERM 15

# Latency

15

Time from question to answer: embed, search, re-rank and generate all **add delay**.

**Fixes**

Cache embeddings, shrink search scope, parallelise calls.

latency.py

```
t0 = time.time(); emb = embed(query)
t1 = time.time(); res = vector_db.search(emb)
t2 = time.time(); out = llm.generate(query, res)
t3 = time.time()
print(t1-t0, t2-t1, t3-t2)
```

**In practice**

Embed 40ms, search 30ms, generate 1.2s. Generate dominates.

> **Measure each stage before you optimise any.**




**Calab**
AI & Tech


Follow me

THE FULL FLOW

# Query → Embed → Search → Filter → Re-rank → Verify

Start simple: chunk, embed, store, retrieve, generate.
Add the rest when quality demands it.

Save Repost Comment

Follow for more AI insights